M-3

① Logic

Algorithm

(i) Read the number of package value (N).
(ii) Store all package values in an array.
(iii) Divide the array into two halves recursively until each subarray contains only one element.
(iv) Merge the divided subarrays by comparing their elements.
(v) While merging
→ Place the smaller element first.
→ continue until both subarrays are merged.
(vi) Repeat the process until the complete array is sorted.
(vii) Print the sorted array.

Question1.java 1

Question1.java > Java > Question1 > mergeSort(int[] arr, int left, int right)

```java
import java.util.Scanner;
public class Question1 {
    static void merge(int arr[], int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];
        for (int i = 0; i < n1; i++) {
            L[i] = arr[left + i];
        }

        for (int j = 0; j < n2; j++) {
            R[j] = arr[mid + 1 + j];
        }

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {

            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
    static void mergeSort(int arr[], int left, int right) {

        if (left < right) {
            int mid = (left + right) / 2;

```

File Edit Selection View Go Run ··· WEEK 3 MODULE

Question1.java 1 ×

Question1.java > Java > Question1

```
2   public class Question1 {
41      static void mergeSort(int arr[], int left, int right) {
49              mergeSort(arr, left, mid);
50              mergeSort(arr, mid + 1, right);
51              merge(arr, left, mid, right);
52          }
53      }
54
        Run | Debug | Run main | Debug main
55      public static void main(String[] args) {
56          Scanner sc = new Scanner(System.in);
57          int n = sc.nextInt();
58          int[] arr = new int[n];
59
60          for (int i = 0; i < n; i++) {
61              arr[i] = sc.nextInt();
62          }
63          mergeSort(arr, left: 0, n - 1);
64          for (int i = 0; i < n; i++) {
65              System.out.print(arr[i] + " ");
66          }
67
68          sc.close();
69      }
70  }
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\COLLEGE MODULE\WEEK 3 MODULE> cd "c:\COLLEGE MODULE\WEEK 3 MODULE\" ; if ($?) { javac Qu
6
450
1200
800
650
300
1500
300 450 650 800 1200 1500
PS C:\COLLEGE MODULE\WEEK 3 MODULE>
```

(2) Logic

Algorithm :-

(i) Read the number of processing times (N)

(ii) Store all processing times in an array.

(iii) Sort the array using merge sort.

(iv) Print the sorted processing times.

(v) Find the median using.

median = arr [n/2].

(vi) count how many processing times are greater than the median.

(vii) Find the difference between the maximum and minimum processing times.

difference = arr [n-1] - arr [0]

(viii) Print the median count and difference.

Question2.java 1

Question2.java > Java > Question2 > mergeSort(int[] arr, int left, int right)

```java
import java.util.Scanner;
public class Question2 {
    static void merge(int arr[], int left, int mid, int right) {

        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {

            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
    static void mergeSort(int arr[], int left, int right) {

        if (left < right) {

            int mid = (left + right) / 2;

```

File Edit Selection View Go Run ··· WEEK 3 MODULE

Question2.java 1

Question2.java > Java > Question2 > mergeSort(int[] arr, int left, int right)

```
2   public class Question2 {
41      static void mergeSort(int arr[], int left, int right) {
48
49              mergeSort(arr, left, mid);
50              mergeSort(arr, mid + 1, right);
51              merge(arr, left, mid, right);
52          }
53  }
        Run | Debug
54      public static void main(String[] args) {
            Run main | Debug main
55          Scanner sc = new Scanner(System.in);
56          int n = sc.nextInt();
57          int[] arr = new int[n];
58
59          for (int i = 0; i < n; i++) {
60              arr[i] = sc.nextInt();
61          }
62          mergeSort(arr, left: 0, n - 1);
63
64          for (int i = 0; i < n; i++) {
65              System.out.print(arr[i] + " ");
66          }
67          System.out.println();
68          int median = arr[n / 2];
69          System.out.println("Median: " + median);
70          int count = 0;
71          for (int i = 0; i < n; i++) {
72              if (arr[i] > median) {
73                  count++;
74              }
75          }
76          System.out.println("Orders Above Median: " + count);
77          int difference = arr[n - 1] - arr[0];
78          System.out.println("Difference: " + difference);
79          sc.close();
80      }
81  }
82
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
7
12
25
18
30
15
22
40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\COLLEGE MODULE\WEEK 3 MODULE>
```

Date
Page

(2)

Logic
Algorithm :-

(i) Read the number of response times (N).
(ii) Store all response times in an array.
(iii) Select the last element as the pivot.
(iv) Rearrange the array so that.

→ Elements smaller than the pivot come before it.
→ Elements greater than the pivot come after it.

(v) Place the pivot in its correct sorted position.
(vi) Recursively apply the same process to the left and right sub arrays.
(vii) Repeat until the entire array is sorted.
(viii) Print the sorted array.

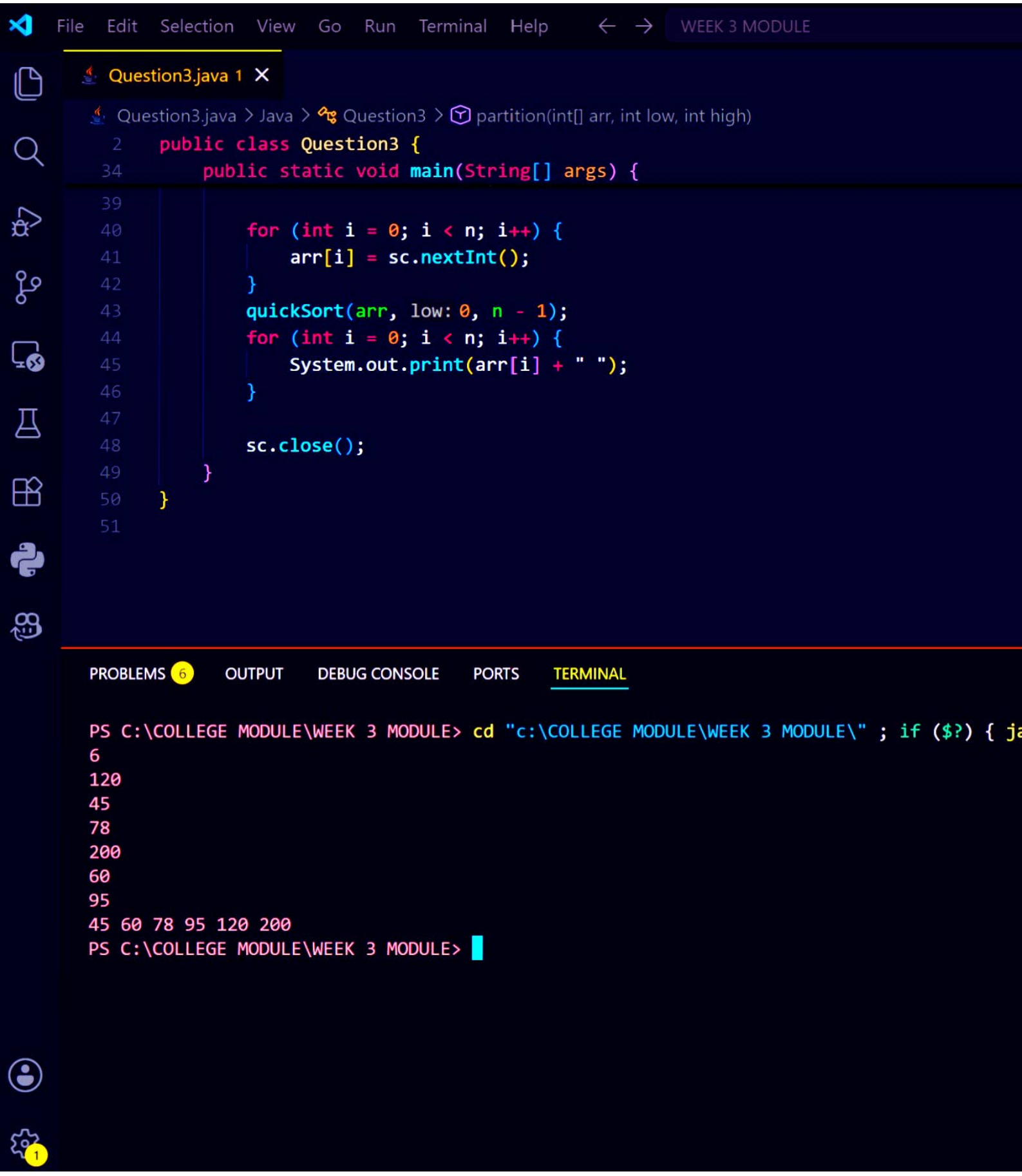
File Edit Selection View Go Run Terminal Help ← → WEEK 3 MODULE

Question3.java 1 ×

Question3.java > Java > Question3 > partition(int[] arr, int low, int high)

```java
import java.util.Scanner;
public class Question3 {
    static int partition(int arr[], int low, int high) {

        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        return i + 1;
    }
    static void quickSort(int arr[], int low, int high) {

        if (low < high) {

            int pivotIndex = partition(arr, low, high);

            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
```

0 6 Indexing completed. Java: Ready


File Edit Selection View Go Run Terminal Help
WEEK 3 MODULE
Question3.java 1
Question3.java > Java > Question3 > partition(int[] arr, int low, int high)
public class Question3 {
public static void main(String[] args) {
for (int i = 0; i < n; i++) {
arr[i] = sc.nextInt();
}
quickSort(arr, low: 0, n - 1);
for (int i = 0; i < n; i++) {
System.out.print(arr[i] + " ");
}
sc.close();
}
}
PROBLEMS 6
OUTPUT
DEBUG CONSOLE
PORTS
TERMINAL
PS C:\COLLEGE MODULE\WEEK 3 MODULE> cd "c:\COLLEGE MODULE\WEEK 3 MODULE\" ; if ($?) { ja
6
120
45
78
200
60
95
45 60 78 95 120 200
PS C:\COLLEGE MODULE\WEEK 3 MODULE>

④

Logic

Algorithm :-

① Read the number of trade values (N)

(ii) Store all trade values in an array and calculate the total sum.

(iii) Sort the array in descending order using Quick sort.

(iv) Print the sorted trade values.

(v) Print the first 5 values

(vi) calculate the average of the Top 5 values.

(vii) Calculate the overall average of all trade values.

(viii) Count how many trade values are greater than the overall average.

(ix) Print the average of the Top 5 values and the count.

File Edit Selection View Go Run ··· WEEK 3 MODULE

Question4.java 1

Question4.java > Java > Question4 > main(String[] args)

```java
public class Question4 {
    static int partition(int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {

            if (arr[j] > pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        return i + 1;
    }
    static void quickSort(int arr[], int low, int high) {

        if (low < high) {

            int pivotIndex = partition(arr, low, high);

            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);
        }
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        long totalSum = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            totalSum += arr[i];
        }
```

File Edit Selection View Go Run ··· WEEK 3 MODULE

Question4.java 1

Question4.java > Java > Question4 > main(String[] args)

```
2   public class Question4 {
36      public static void main(String[] args) {
49
50          quickSort(arr, low: 0, n - 1);
51          for (int i = 0; i < n; i++) {
52              System.out.print(arr[i] + " ");
53          }
54          System.out.println();
55          System.out.print(s: "Top 5: ");
56
57          long topSum = 0;
58
59          for (int i = 0; i < 5; i++) {
60              System.out.print(arr[i] + " ");
61              topSum += arr[i];
62          }
63          System.out.println();
64          long topAverage = topSum / 5;
65          System.out.println("Average of Top 5: " + topAverage);
66          double overallAverage = (double) totalSum / n;
67          int count = 0;
68
69          for (int i = 0; i < n; i++) {
70              if (arr[i] > overallAverage) {
71                  count++;
72              }
73          }
74
75          System.out.println("Values Above Overall Average: " + count);
76
77          sc.close();
78      }
79  }
```

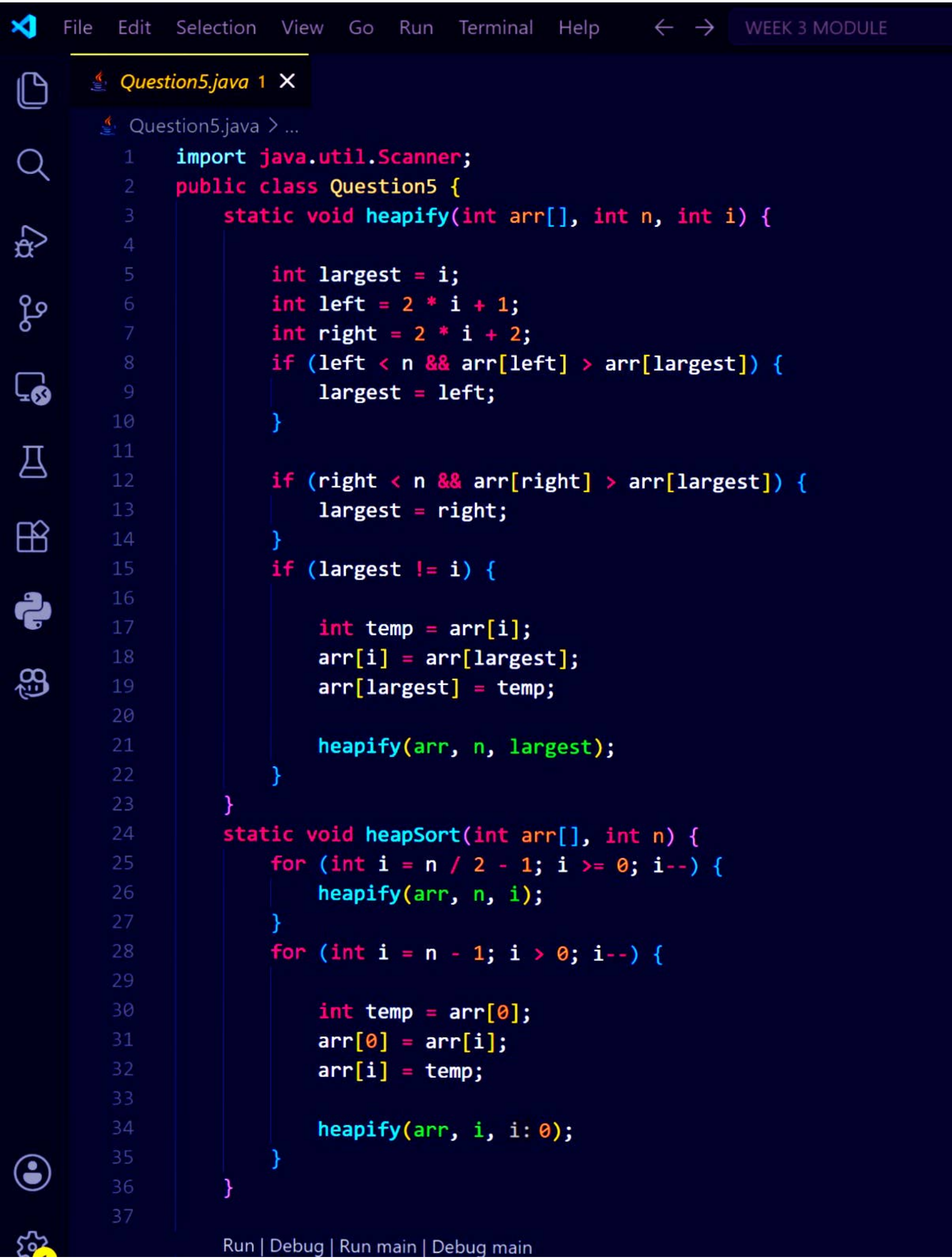
PROBLEMS 6 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
8
500
1200
700
2000
900
1500
3000
2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
```

⑤ Logic

Algorithm :-

(i) Read the number of player scores (N).
(ii) Store all scores in an array.
(iii) Build a max Heap from the array.
(iv) Swap the root with the last element.
(v) Reduce the heap size by one
(vi) Heapify the remaining heap.
(vii) Repeat until all element are sorted.
(viii) Print the sorted scores in ascending order.


File Edit Selection View Go Run Terminal Help
WEEK 3 MODULE
Question5.java 1
Question5.java > ...
import java.util.Scanner;
public class Question5 {
static void heapify(int arr[], int n, int i) {
int largest = i;
int left = 2 * i + 1;
int right = 2 * i + 2;
if (left < n && arr[left] > arr[largest]) {
largest = left;
}
if (right < n && arr[right] > arr[largest]) {
largest = right;
}
if (largest != i) {
int temp = arr[i];
arr[i] = arr[largest];
arr[largest] = temp;
heapify(arr, n, largest);
}
}
static void heapSort(int arr[], int n) {
for (int i = n / 2 - 1; i >= 0; i--) {
heapify(arr, n, i);
}
for (int i = n - 1; i > 0; i--) {
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
heapify(arr, i, i: 0);
}
}
Run | Debug | Run main | Debug main

File Edit Selection View Go Run Terminal Help

Question5.java 1

Question5.java > ...

```
2   public class Question5 {
38      public static void main(String[] args) {
39
40          Scanner sc = new Scanner(System.in);
41          int n = sc.nextInt();
42
43          int[] arr = new int[n];
44          for (int i = 0; i < n; i++) {
45              arr[i] = sc.nextInt();
46          }
47          heapSort(arr, n);
48          for (int i = 0; i < n; i++) {
49              System.out.print(arr[i] + " ");
50          }
51
52          sc.close();
53      }
54  }
55
56
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\COLLEGE MODULE\WEEK 3 MODULE> cd "c:\COLLEGE MODULE\WEEK
6
850
1200
950
600
1500
1000
600 850 950 1000 1200 1500
PS C:\COLLEGE MODULE\WEEK 3 MODULE>
```

⑥ Logic

Algorithm

(i) Read the number of responding times (N)

(ii) Store all response times in an array and calculate the total sum.

(iii) Sort the array in ascending order using Heap sort.

(iv) Print the sorted response time.

(v) The first element in the fastest responses time.

(vi) The last element is the slowest response time.

(vii) Calculate the average -

Average = Sum / N.

(viii) Count how many response times are less than the average.

(ix) Calculate the percentage.

$$\text{Percentage} = (\text{Count} \times 100) / N$$

(x) Print all required results.

File Edit Selection View Go Run Terminal Help WEEK 3 MODULE

Question6.java 1

Question6.java > Java > Question6 > main(String[] args)

```java
import java.util.Scanner;
public class Question6 {
    static void heapify(int arr[], int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }

        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }

        if (largest != i) {

            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }
    static void heapSort(int arr[], int n) {
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        for (int i = n - 1; i > 0; i--) {

            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            heapify(arr, i, i: 0);
        }
    }

```

File Edit Selection View Go Run ··· WEEK 3 MODULE

Question6.java 1

Question6.java > Java > Question6 > main(String[] args)

```java
public class Question6 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];



        int sum = 0;
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }
        heapSort(arr, n);
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
        System.out.println("Fastest: " + arr[0]);

        System.out.println("Slowest: " + arr[n - 1]);

        double average = (double) sum / n;
        System.out.println("Average: " + average);

        int count = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] < average) {
                count++;
            }
        }
        System.out.println("Cases Faster Than Average: " + count);
        double percentage = (double) count * 100 / n;

        System.out.printf(format: "Percentage: %.2f%%", percentage);

        sc.close();
    }
}
```

PROBLEMS 6 OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
PS C:\COLLEGE MODULE\WEEK 3 MODULE> cd "c:\COLLEGE MODULE\WEEK 3 MODULE\" ; if ($?) { javac Question6.
8
12
7
15
9
20
5
10
8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\COLLEGE MODULE\WEEK 3 MODULE>
```